CS50x 2026 - Lecture 2 Notes - Arrays

By Wisdom

The process of turning a C program into an executable happens in **four main stages**:

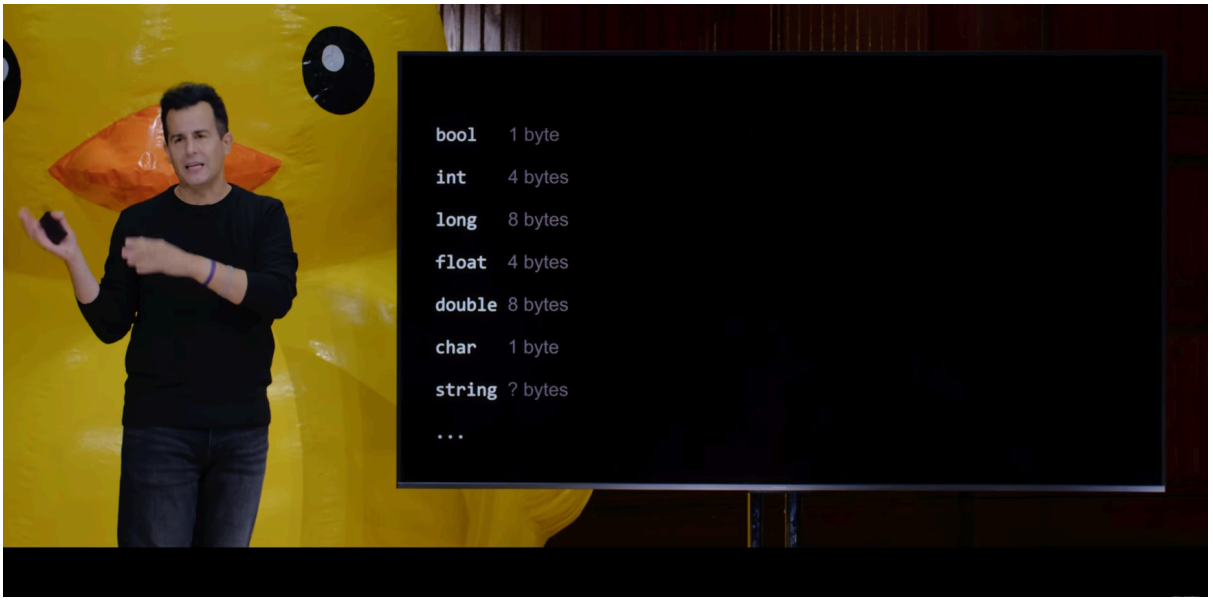
1. **Preprocessing** – The preprocessor handles lines that start with `#`, such as `#include`, `#define`, and macros. It also removes comments and expands macros before the actual compilation begins.
2. **Compiling** – The compiler checks the code for syntax errors and converts the preprocessed C code into **assembly code**.
3. **Assembling** – The assembler then converts the assembly code into **machine code**, producing object files (usually with a `.o` extension).
4. **Linking** – Finally, the linker combines the object files with any required libraries to produce the **final executable program**.

In short, the C compilation pipeline looks like this:

C source code → Preprocessing → Compilation → Assembling → Linking → Executable program.

Data Types in C

Data types in C and how many bytes they take up in the computer memory



```c
#include <stdio.h>

#define NUM_SCORES 3

int main(void) {
    int scores[NUM_SCORES];

    for (int i = 0; i < NUM_SCORES; i++) {
        printf("Enter score %d: ", i + 1);
        scanf("%d", &scores[i]);
    }

    int sum = 0;
    for (int i = 0; i < NUM_SCORES; i++) {
        sum += scores[i];
    }

    printf("The average score is: %.2f\n", (float)sum / NUM_SCORES);
}
```

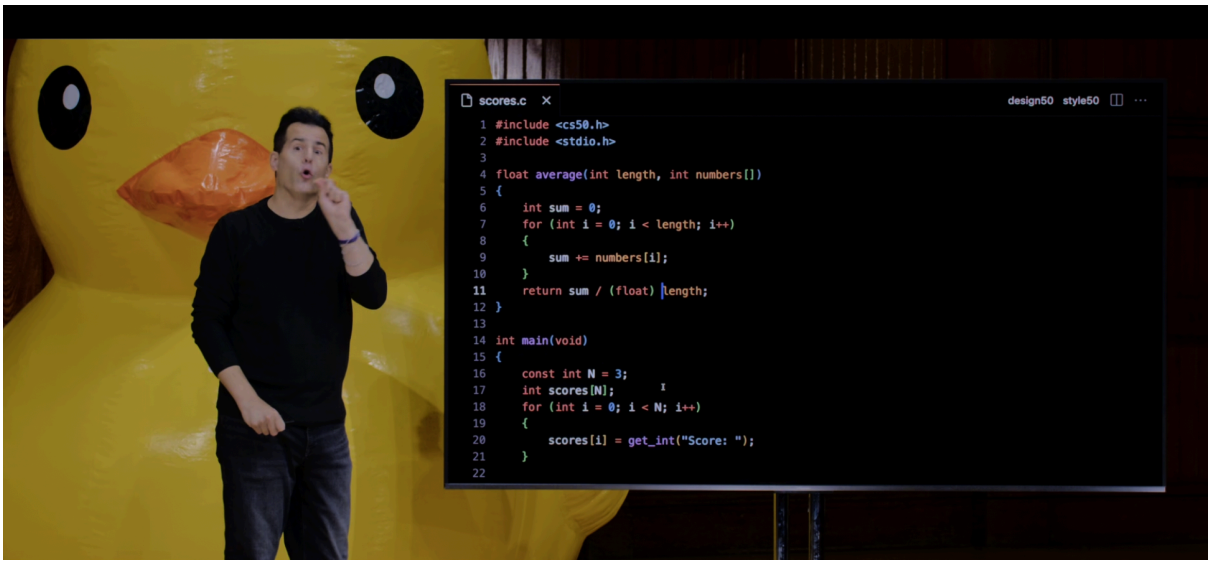
```
Apple@MacBook-Pro Learning C % gcc avg.c -o avg
Apple@MacBook-Pro Learning C % ./avg
The average score is: 90
Apple@MacBook-Pro Learning C % gcc avg2.c -o avg2
avg2.c:5:16: warning: variable length array folded to constant array as an extension [-Wgnu-folding-constant
]
    5 |     int scores[NUM_SCORES];
      |                ^~~~~~~~~~
1 warning generated.
Apple@MacBook-Pro Learning C % gcc avg2.c -o avg2
Apple@MacBook-Pro Learning C % ./AVG2
Enter score 1: 234
Enter score 2: 345
Enter score 3: 33
The average score is: 204.00
Apple@MacBook-Pro Learning C %
```

Working through this program helped me understand several core ideas in C programming: arrays, loops, user input, and basic data processing. Instead of creating separate variables like `score1`, `score2`, and `score3`, I used an **array** (`scores`) to store multiple values of the same type. I also used a **loop** to repeatedly ask the user for input and store each score in the array. I found this design elegant because it scales easily—if I wanted to collect 10 or even 100 scores, I would only need to change the constant defining the array size, and the rest of the program would still work.

I also learned how to process data stored in an array. After collecting the scores, I used another loop to iterate through them and compute the total using `sum += scores[i]`. This accumulation pattern made sense to me because it systematically adds each value in the array to a running total. Once the sum is calculated, I divide it by the number of scores to compute the average, casting the sum to a `float` so the result includes decimals instead of being truncated by integer division.

Finally, I ran into a compiler warning that helped me understand something deeper about how C is designed. Even though I declared `const int NUM_SCORES = 3 in the first version of my code`, the compiler still treats it as a runtime variable rather than a true compile-time constant. That's why using a **macro like `#define NUM_SCORES 3`** removes the warning—it replaces the value before compilation so the compiler sees `int scores[3]`. This taught me that C deliberately separates compile-time constants from runtime variables, reflecting its low-level design, in which the programmer has very explicit control over memory and how programs are compiled.

Arrays



Arrays in C for me is still some form of a hard concept to learn especially when it gets pretty complex. So what is an Array? An array is simple a variable which has the capacity to hold multiple values in itself like a container that can house an unlimited number of balls if not set.

As well, arrays can be capped to the exact number of values you wanna store in it like a conatiner which can only house 5 balls,

Code example of an array in action where it is used to calculate the average mark of student scores in an example. This was quite difficult to wrap my head around.

More so, Through this exercise I learned several core concepts in C programming. I learned how to use **arrays** to store multiple values of the same type instead of creating many separate variables, and how **loops** make it easy to repeatedly collect input from a user and process data. By iterating over the array, I can accumulate values using patterns like `sum += marks[i]`, which is a common way to compute totals before calculating something like an average. I also learned the importance of **type casting** (like `(float)sum`) to avoid integer division when I want decimal results.

I also discovered how **macros work in C** and why they are used for constants. Using `#define` creates a compile-time constant through text substitution before the compiler runs, which is why it works better for things like array sizes. I learned that macros should not include semicolons and should not reuse names that will later be used as variables, since the preprocessor literally replaces the text. Overall, this helped me understand not just how to write a small program that collects marks and computes an average, but also how C's design gives the programmer explicit control over memory, compilation, and program structure.

```c
C task.c > main(void)
#include <stdio.h>

#define NUM_MARKS 5

int main(void) {

    int marks[NUM_MARKS];

    for (int i = 0; i < NUM_MARKS; i++) {
        printf("Enter mark %d: ", i + 1);
        scanf("%d", &marks[i]);
    }

    int sum = 0;
    for (int i = 0; i < NUM_MARKS; i++) {
        sum += marks[i];
    }

    float average = (float)sum / NUM_MARKS;

    printf("Average mark: %.2f\n", average);

    return 0;
}
```

From this exercise, I learned how functions work more clearly in C and that a function does not have to start with `void`. The first word in a function definition is simply the **return type**, so a function can return different types like `int`, `float`, or `char`. For example, `float average(int length, int numbers[])` returns a decimal value after computing the average of an array, which also taught me how arrays can be passed into functions and iterated over with a loop to accumulate values like `sum += numbers[i]`. I also learned why **type casting,** such as `(float) length,` is important to avoid integer division and produce a correct decimal result.

I also understood when `void` is actually used in real programs. `void` is used when a function's job is to **perform an action rather than return a value**, such as printing output, updating data through pointers, or running steps in a process like a game loop or system routine. This helped me see that C functions generally fall into two categories: functions that **compute and return values** (like calculating an average) and functions that **perform tasks or side effects** without returning anything. Understanding this distinction made the structure and design of C programs much clearer to me.

Characters & Strings In C

Characters in C double as both the characters themselves and the integers they are derived from.

A string is a paragraph of text. It is an array of characters.

Through this exercise, I learned how strings and characters work in C and how text can be manipulated at a low level. I learned that C does not have a built-in string type; instead, a string is simply an **array of characters (`char`) that ends with a null terminator `\0`**. I also practiced reading input character by character using functions like `getchar()` and printing characters with `%c`. This helped me understand that characters in C are essentially **numbers under the hood**, which makes it possible to manipulate them directly.

I also learned two ways to convert letters to uppercase. One approach uses the standard library function `toupper()` from `<ctype.h>`, which handles the conversion safely. The more "raw" approach relies on the **ASCII representation of characters**, where lowercase and uppercase letters differ by 32. By checking if a character is between `'a'` and `'z'` and subtracting 32, I can manually convert it to uppercase. This showed me how C allows direct control over data and memory, and how many higher-level library functions are built on top of simple numerical operations on characters.

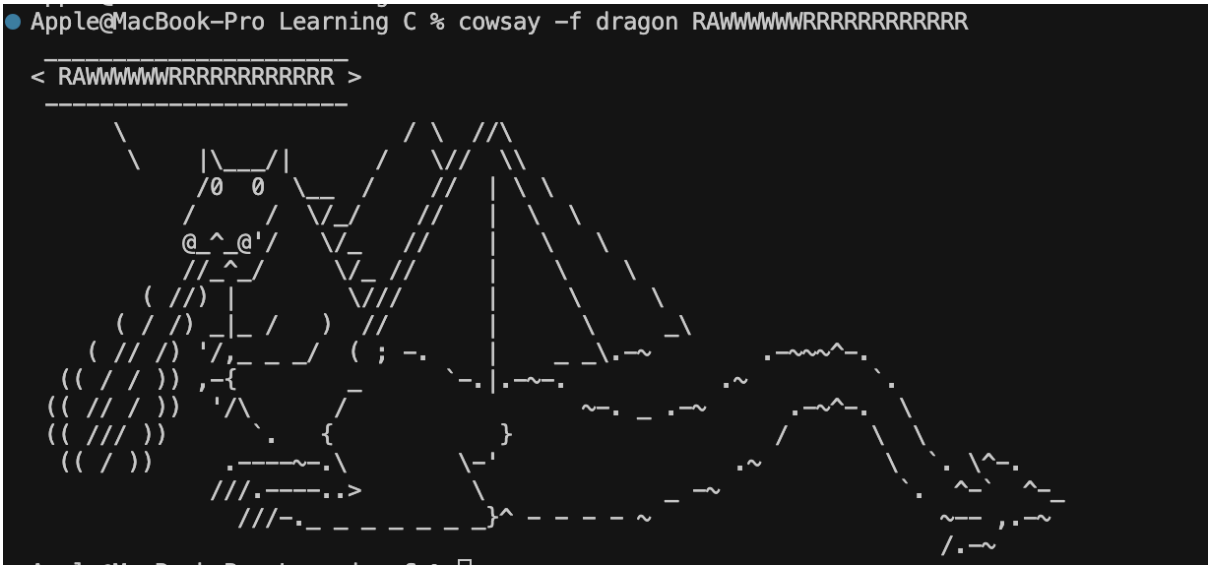
Command Line Arguments in C

```c
C test.c > main(int, char * [])
#include <stdio.h>

int main(int argc, char *argv[])
{
    if (argc < 2)
    {
        printf("hello, world\n");
        return 0;
    }

    printf("hello, ");
    for (int i = 1; i < argc; i++)
    {
        printf("%s", argv[i]);
        if (i < argc - 1) printf(" ");
    }
    printf("\n");
}
```

From this exercise, I learned how **command-line arguments** work in C. When I run a program like `./greet David Malan`, the operating system doesn't just magically pass the words; it gives my program a **list of strings**. `argc` tells me **how many words are in the list**, and it always counts the program name itself as the first word. `argv` is the **array of strings** holding those words, where `argv[0]` is always the program name, `argv[1]` is the first argument typed by the user, and so on. This is why `argc` can be 1 even if the user typed nothing; the program name is always counted.

I also learned how to use this in practice. By checking `argc`, I can decide whether the user typed any arguments and provide a default behavior like printing `hello, world` if none were given. Using a `for` loop starting at `i = 1` allows me to **skip the program name** and only process the user's input. I saw how loops, arrays, and conditionals work together to build flexible programs that can react to what the user types when launching the program. Overall, this taught me how C gives me **full control over the words typed at the terminal** and how to handle them safely and logically in my programs.

I also learned that after a program runs, typing `echo $?` in the terminal prints the program's **exit code**, which tells whether it finished successfully (`0`) or encountered an error (any other number)



Using cowsay for fun for the fundamentals of argc and argv.

Cryptography

Cryptography is the science of **keeping information secret and secure**. At its core, it transforms readable data, called **plaintext**, into a scrambled format called **ciphertext** so that only someone with the correct key can read it. One of the simplest concepts in cryptography is **encryption**, which hides the data, and **decryption**, which recovers it. This ensures that even if someone intercepts the message, they cannot understand it without the key.

There are two main types of cryptography: **symmetric** and **asymmetric**. In symmetric cryptography, the same key is used to encrypt and decrypt a message, so both the sender and receiver must keep the key secret. In asymmetric cryptography, there are **two keys**: a public key to encrypt messages and a private key to decrypt them, which allows secure communication without sharing a secret key in advance. Hashing is another important concept; it converts data into a fixed-length value that is practically impossible to reverse, often used to verify integrity or store passwords securely.

Cryptography is everywhere in real life. It secures online banking, messaging apps, and digital signatures. Learning the basics also teaches important ideas like **confidentiality** (keeping data secret), **integrity** (ensuring data hasn't been changed), and **authentication** (verifying who is sending or receiving data). Understanding these principles helps anyone build more secure systems or evaluate the security of existing ones, and it's the foundation of modern digital security.

```c
#include <stdio.h>
#include <ctype.h>

// Encrypt or decrypt using a simple Caesar cipher
char shift_char(char c, int key) {
    if (isalpha(c)) { // Only shift letters
        char base = isupper(c) ? 'A' : 'a';
        return (c - base + key + 26) % 26 + base; // +26 ensures positive modulo
    }
    return c; // Non-letters remain unchanged
}

int main(void) {
    char text[] = "Hi!"; // Original message
    int key = 1;          // Shift letters by +1

    printf("Original: %s\n", text);

    // Encrypt
    for (int i = 0; text[i] != '\0'; i++) {
        text[i] = shift_char(text[i], key);
    }
    printf("Encrypted: %s\n", text);

    // Decrypt (invert the key)
    key = -key;
    for (int i = 0; text[i] != '\0'; i++) {
        text[i] = shift_char(text[i], key);
    }
    printf("Decrypted: %s\n", text);

    // Special example: ROT13 (key = 13)
    char rot13_text[] = "Hello, world!";
    key = 13;
    for (int i = 0; rot13_text[i] != '\0'; i++) {
        rot13_text[i] = shift_char(rot13_text[i], key);
    }
    printf("ROT13: %s\n", rot13_text);

    return 0;
}
```

Cryptography can be understood simply as **shifting letters to hide a message**. For example, if I have the message “Hi!” and use a key of `+1`, each letter moves one step forward in the alphabet: `H` becomes `I` and `i` becomes `j`. Symbols like `!` don’t change, so the encrypted message becomes “Ij!”. This is called a **Caesar cipher**, one of the simplest types of encryption. To decrypt it and get back the original message, I just **invert the key** (use `-1`), which moves each letter back to where it started.

There’s also a special case called **ROT13**, where the key is `13`. Since the alphabet has 26 letters, shifting by 13 twice brings letters back to their original positions. For example, `A` → `N` after the first ROT13, and `N` → `A` after the second. This shows a neat trick in cryptography:

some shifts are their own inverse, which makes encoding and decoding very simple. Overall, this teaches the basic ideas of **encryption, decryption, and how keys control the transformation of messages**.